Journal of Systems Architecture 175 (2026) 103780

Contents lists available at ScienceDirect

Journal of Systems Architecture

journal homepage: www.elsevier.com/locate/sysarc

DCoL-A: Agentic dual chain of thinking helps LLMs pretend logic solvers☆

Minyu Chen [a], Ling-I Wu [a], Ruibang Liu [a], Xi Chang [b], Jianxin Xue [b], Guoqiang Li [a,*]

[a] Shanghai Jiao Tong University, Shanghai, 200240, China
[b] Shanghai Polytechnic University, Shanghai, 201209, China

ARTICLE INFO

Keywords:
Logic solver
Large language models
Dual chains of logic
SAT problem

ABSTRACT

Logical solvers are typically employed for the static analysis and formal verification of functional code that is not amenable to verification through execution. Concurrently, some research efforts aim to convert natural language into logic code to bolster reasoning capabilities with the aid of logical solvers. However, the transformation process from ambiguous natural language to a precise logical framework is error-prone and costly. Nevertheless, LLMs may introduce misalignment and formatting errors that are unacceptable for logical solvers. Considering the strong logical reasoning capabilities of LLMs, and their robustness to inputs with minor syntactic errors or non-standard theoretical constraints, we propose utilizing LLMs to emulate logical solvers for logic code simulation. To address the tendency of LLMs to follow a single, flawed reasoning path, we introduce a novel LLM-based code simulation technique called Dual Chains of Logic (DCoL), along with its evolution into a more robust, agent-based framework. Additionally, we have compiled several new datasets specifically for this task, covering a range of functional, semantic, and scalability benchmarks, and conducted comprehensive evaluations on a range of LLMs, from foundational models to state-of-the-art reasoning agents. Our assessments demonstrate that the proposed frameworks significantly enhance the performance of modern LLMs, with the agent-based approach proving particularly effective on these computationally hard logic problems.

1. Introduction

Logic serves as the foundation for the majority of formal methodologies in software engineering. By translating specifications, constraints, and test cases into logical frameworks like First Order Logic (FOL) or Satisfiability Modulo Theories (SMT), logic solvers can check the satisfiability of complex problems. Thus, state-of-the-art solvers including Z3 [1] and CVC5 [2] are applied in wide aspects of software engineering, including software verification [3], software testing [4], program synthesis [5], and program analysis [6]. However, logic remains a complex subject within human cognition, presenting several challenges to the deployment of logic solvers in software engineering. One key obstacle is the gap between natural language (NL) and the formal Solver Language (SL), such as the SMT-LIB format, used to express logical problems. This gap impedes not only the programming process but also the comprehension of code segments within existing software systems. Furthermore, the integration of logic in software engineering often involves various theories such as arrays, integers, reals, strings, and bit-vectors. This diversity has led to the development of specialized solvers [6, 7] tailored to different logical theories. While these specialized solvers are highly effective within their defined theoretical boundaries, they exhibit significant fragility when encountering the inherent noise of human intent or the complex constraints common in real world software development. In this paper, we explore the capacity of Large Language Models to internalize logical semantics through the autonomous simulation of logic code. We introduce the Dual Chains of Logic (*DCoL*) framework and its agentic version *DCoL-A* as a structured reasoning architecture based on large language models (LLMs) for simulating the execution semantics of constraint based logic programs integrated into general purpose programming environments. By decoupling the satisfying search and contradictory proof, our approach leverages the functional asymmetry of formal verification to enhance the reliability of neural logic simulation.

Recent advances in LLMs such as GPT [7] and LLaMA [8] have showcased their ability to perform logic reasoning akin to humans [9, 10]. By incentivizing LLMs to translate natural language into solver languages with in-context learning (ICL) [7] or model fine-tuning [11], LLMs can successfully solve simple propositional logic questions in NL form [12,13]. When faced with intricate logic-based queries, such as those involving FOL [14] or the SAT decision problem [15], LLMs exhibit challenges in direct problem-solving and show low successful execution rates of code generation. LLMs also fail to conduct relational

☆ This article is part of a Special issue entitled: 'SETTA 2024' published in Journal of Systems Architecture.

* Corresponding author.
E-mail address: li.g@sjtu.edu.cn (G. Li).

https://doi.org/10.1016/j.sysarc.2026.103780
Received 6 June 2025; Received in revised form 12 February 2026; Accepted 11 March 2026
Available online 12 March 2026
1383-7621/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

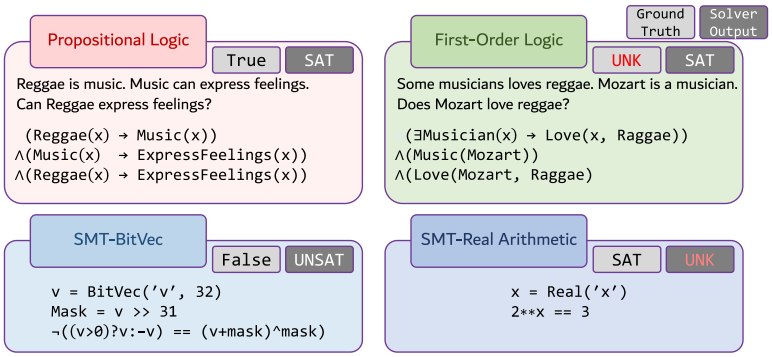


Fig. 1. Samples of logical problem to be studied in our research. The **light** gray boxes display the ground truth of the given problems, while the results given by the SMT solver are presented in the **dark** gray boxes. 'UNK' denotes unknown.

reasoning [16]. Nonetheless, it is crucial to recognize the valuable bridge LLMs establish between NL and SL.

In this work, we focus on a distinct but related challenge: the simulation of logic code. We define "logic code" as an executable intermediate representation that bridges the gap between ambiguous natural language descriptions and precise, formal logical problems. It is neither pure natural language nor pure formal logic, but rather a structured artifact used to describe and solve logical problems. For example, Z3Py embeds a domain-specific language for SMT solving within the Python programming language, making it directly executable by an interpreter. By utilizing LLMs as simulator, we define logic code simulation as the task of predicting the final output of such code by analyzing the logical essence without executing the underlying solver.

This task is not intended to replace highly optimized, traditional solvers. Instead, it explores a different set of capabilities where LLMs offer unique advantages. First, LLMs demonstrate inherent semantic robustness to inputs that would cause traditional parsers to fail. They can correctly infer the logic despite syntax errors or malformed code, such as misspelled variables or mismatched parentheses, allowing them to handle higher level abstractions without needing immediate repair. Second, LLMs achieve theory transcendence by leveraging their world knowledge to reason about problems that fall outside a solver theoretical framework, such as non polynomial constraints like $2^x == 3$ for which a solver might return unknown. These observations motivate our central research question: Can language models act as effective logic code simulators, and what methodologies can enhance their performance in this specialized domain? To investigate this question, we first embarked on an empirical study to assess current LLM capabilities and challenges in this domain. Our preliminary assessments revealed that while foundational aptitude exists, performance is hindered by a key shortcoming: LLMs are prone to getting stuck in a single, incorrect line of reasoning when using standard prompting techniques. The insight that this single path reasoning was a primary failure mode suggests the need for a more structured reasoning mechanism that forces the model beyond its initial biased output.

To address this identified shortcoming, the DCoL framework is designed to specifically mitigate the single path reasoning failure mode by leveraging the problem's typical binary outcome of SAT or UNSAT. Instead of a single chain of thought, DCoL prompts the LLM to explore two distinct reasoning paths based on contrary hypotheses: one assuming the *logic code* will result in `SAT` and the other assuming `UNSAT`. The LLM is guided to conduct deep reasoning along each path, for instance, by attempting to find potential satisfying assignments for the `SAT` hypothesis or identifying conflicting constraints for the `UNSAT` hypothesis. This structured approach mirrors the functional asymmetry of formal verification where the framework requires the LLM to integrate and evaluate the logical soundness of both paths. It must determine which hypothesis is supported by a valid argument and which path's reasoning is flawed, thereby selecting the final outcome. We emphasize that DCoL is not a new combinatorial solving algorithm like DPLL. Its novelty lies in its role as a multi-agent reasoning framework that forces the model to explore and evaluate contrary hypotheses rather than settling on the first potentially flawed chain of thought.

In this paper, we experimentally demonstrate DCoL's enhanced performance in logic code simulation. We also investigate the scalability of LLMs in this task and discuss their theoretical properties as solvers. Furthermore, we explore the broader benefits that LLMs, particularly when augmented with DCoL, can bring to logic solving, including their tolerance for slips or minor errors in logic codes.

We highlight our contributions as follows:

- We introduce the novel task of Logic Code Simulation and compile several datasets to systematically evaluate the performance of LLMs.
- We propose a novel reasoning framework, DCoL, and its advanced agent-based version, DCoL-A, which leverages specialized Searcher, Prover, and Judger agents to simulate the functional asymmetry of formal verification.
- We conduct extensive evaluations on a range of LLMs, from foundational to state-of-the-art reasoning models. Our results show the proposed agent-based framework achieves state-of-the-art performance, with up to 88.5% accuracy on the challenging 3SATSIM dataset.

This paper is an extension of a previously published conference paper published in SETTA 2024 [17]. The main difference lies in several places in this version.

- **Evaluation of New-Generation LLMs**. With the ongoing advancements in LLMs, new models with enhanced reasoning capabilities, such as Gemini and DeepSeek-R1, have been introduced to tackle specialized logical problems.
- **Upgrade of DCoL to an Agent-Based Framework**. To overcome limitations in the original prompt-based DCoL, specifically issues related to context length overflow and interference between the reasoning paths, we re-engineer DCoL into an agent-based system. In this upgraded architecture, the constructive search for assignments and the deductive proof of contradictions are handled by independent agents, thereby utilizing the search and verification asymmetry of SAT problems. A third judger agent then analyzes the outputs from these two paths to determine the final conclusion. This architectural enhancement has yielded significant performance improvements across all evaluated tasks.
- **A New 3SATSIM Dataset**. We propose the 3SATSIM dataset incorporating detailed statistical features for each problem instance, including metrics like the number of variables and clauses. Consequently, 3SATSIM now presents more complex problem structures

and a higher level of reasoning difficulty compared to the Z3Tutorial and Z3Test datasets. This enriched dataset enables a more systematic assessment of LLM performance on professional-grade SAT problems and allows for a more in-depth investigation into their scalability when faced with increasingly complex problems.

The remainder of this paper is organized as follows. Section 2 reviews the background on logical solvers and LLMs, along with related work in code simulation. Section 3 introduces our proposed DCoL framework and its evolution into an agent-based architecture. Section 4 describes the experimental setup, including the models evaluated, the datasets we curated, and the baseline methods . Section 5 presents a comprehensive evaluation of our approach, analyzing functional performance, robustness, and scalability on complex problems. Finally, Section 6 concludes the paper and discusses potential avenues for future research.

2. Background and related work

Large Language Model. LLMs are generative models based on the pre-trained Transformer architecture. Most LLMs utilize a generative model architecture, where given a sentence of n tokens, the model is trained to maximize the likelihood of the ground-truth token t_i at the current time step t based on its preceding sequence $t_{i-1}, \ldots, t_1$. The training of LLMs typically follows three main processes: unsupervised training on large amounts of unlabeled text data without explicit human annotations, supervised fine-tuning on labeled data relevant to specific tasks or domains, and reinforcement learning on feedback from human annotators or evaluators. Leveraging extensive multimodal data and employing pre-training and fine-tuning techniques, LLMs have demonstrated state-of-the-art performance across various downstream tasks, such as machine translation, numerical reasoning, and code clone detection, using either few-shot learning with minimal examples or zero-shot prompting with task-specific instructions.

Logical Solver. We begin by identifying logical problems to be addressed in this work. Generally speaking, a key logical problem in formal methods underlies the framework of SMT. SMT is a variation of the SAT problem for FOL, with the interpretation of symbols constrained by specific theories, such as real arithmetic and bit vectors. Our study encompasses several subsets of SMT problems, including propositional logic, SMT problems themselves, and a fragment of first-order logic within the solving ability of SMT, along with external theories defining constraints. Thus, we focus on SMT-based languages and solvers, to encode and solve those logical problems in a unified manner.

We elaborate on the samples in Fig. 1 to illustrate the scope of problems studied. Often, the task involves transforming a judgment problem *Is hypothesis H True, False, or Uncertain?* into a satisfiability check. We map these outputs using an SMT solver as follows: *True* corresponds to the premises plus the **negation** of H being UNSAT; *False* corresponds to the premises plus the **affirmation** of H being UNSAT; and *Uncertain* means neither is entailed, corresponding to both checks returning SAT. For the propositional logic example, the proposed hypothesis can be derived, aligning with the *True* case. FOL supports quantifiers ∀ and ∃, allowing for more expressive statements. In the FOL case in Fig. 1, the solver's result is indeterminate (UNK), which aligns with our *Uncertain* mapping (i.e., both checks would return SAT). Concerning SMT-based problems, theories also contribute to their complexity. In the SMT-BitVec case, the target is to ascertain equality for an expression involving bit-vector operations. Here, the True/False output corresponds directly to SAT/UNSAT. Conversely, the SMT-Real-Arithmetic example is drawn from the Z3py manual. The Z3 solver produces an *unknown* result for $2^x == 3$, as this problem is non-polynomial. In contrast, individuals can readily find a solution as $x = log_2 3$. This highlights a key motivation for our work: exploring how LLMs might simulate logic in scenarios where traditional solvers are limited.

Code Simulation with LLMs. Code simulation is a relatively new topic of interest. It is defined as predicting the concrete outputs of codes aligning with the execution results. Recent studies on Turing Completeness of transformers and LLMs [18–21] have suggested that LLMs are capable of interpreting instructions from code, showing potential to simulate the execution of code and algorithms as analog models. The previous works encourage LLMs to simulate a variety of tasks, such as keyboard and mouse actions on computers [22] and optimizers [23]. While transformer-based models are trained to predict the execution traces of codes [24] and UNSAT cores [25], a recent attempt at simulating code with LLMs has been proposed [26]. Their results indicate that LLMs can execute instructions sequentially.

Our proposed logic code simulation framework differs from this general-purpose code simulation. It focuses on simulating specialized codes for logical solvers rather than general-purpose code. The potential outcome space is constrained to either SAT or UNSAT, which makes accurate predictions more attainable, as logical codes can be more clearly expressed and understood in natural language.

3. DCoL: Dual Chains of Logic

In this section, we present a new prompt technique, *Dual Chains of Logic* (DCoL), aiming to improve the accuracy, reasoning process, and robustness of the logic code simulation task.

3.1. A motivating example

For complex reasoning problems, the specific reasoning path taken is crucial, as often only a few paths lead to the correct solution. While general problem-solving tasks, such as mathematical computation or text generation, contend with a virtually infinite search space that complicates the construction of precise reasoning paths, logic problems amenable to solvers typically present a dychotomous outcome: SAT or UNSAT. Humans employ diverse reasoning strategies to determine the satisfiability of a set of constraints:

(1) If a problem is presumed to be SAT, a common approach is to find a specific assignment of variables that satisfies all constraints. Consider an illustrative example: given constraints $y \equiv x + 1$ and $\forall y'[(y' \leq 0) \implies (x > y')]$. If we test the assignment $x = 2, y = 3$, it satisfies the first constraint $y \equiv x + 1$. For the second constraint, $\forall y'[(y' \leq 0) \implies (x > y')]$, the variable y' is an **independent variable**,[1] distinct from the y in the first constraint. With $x = 2$, this assignment also meets the second constraint, thus demonstrating satisfiability.

(2) If a problem is presumed to be UNSAT, the goal is to identify a minimal set of conflicting constraints, known as the UNSAT core, where a conflict invariably exists. In the specific case illustrated in Fig. 2, the *IF UNSAT* reasoning path exemplifies a typical logical error. The model attempts to find a contradiction by substituting the constraint $y = x + 1$ from the first line into the second ∀ constraint. This substitution itself is logically invalid. The variable y within the ∀ constraint is a new, universally quantified variable, locally scoped to that expression. It is distinct from the free variable y in the first constraint $y = x + 1$. This erroneous substitution leads the model to derive a false contradiction, as shown in the wrong reasoning path.

However, LLMs often struggle to inherently grasp and navigate these dual implicit reasoning chains for logic problems within a single execution pass. Standard Chain-of-Thought (CoT) prompting, for

[1] Independent variable y' can be replaced with any symbol, such as $\forall z[(z \leq 0) \implies (x > z)]$.

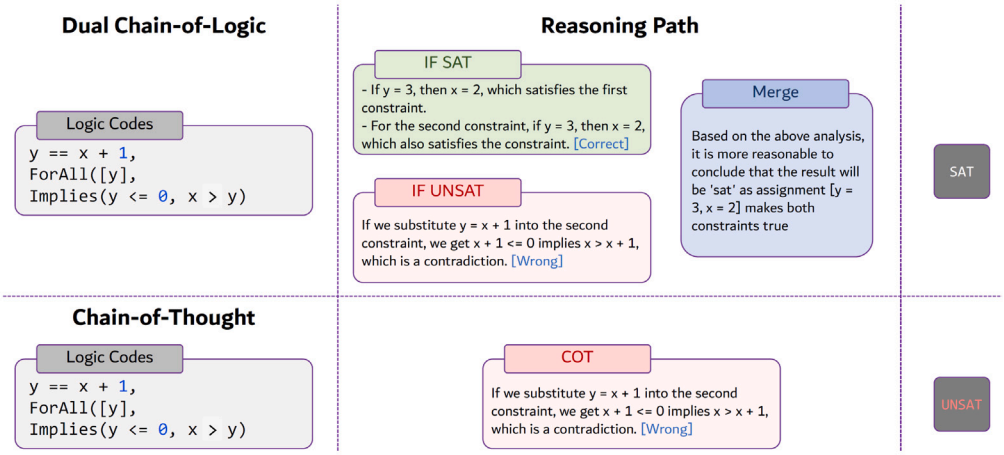


Fig. 2. Overview of the DCoL method: DCoL offers two hypotheses, `SAT` (satisfiable) and `UNSAT` (unsatisfiable), for logic code simulation. The LLMs verify these hypotheses individually before combining them to reach the final decision, while the COT method only outputs one possible reasoning path.

CoT Prompt

```
Suppose you are one of the greatest mathematicians, logician, and programmer. Given the following code in Z3 solver, you are asked to determine the output of the code with #sat or #unsat. Let's think step by step.

Code: [[CODE]]
```

DCoL Prompt

```
Given the following code in Z3 solver, you are asked to determine the output of the code with #sat or #unsat. Follow instructions below:

1. List and understand variables and constraints added to the solver.

2. Based on the constraints, if I say the result is 'sat', is it correct? Can you find satisfied assignments for variables? Let's think step by step.

3. Based on the constraints, if I say the result is 'unsat', is it correct? Can you find conflict constraints? Let's think step by step.

4. Which is more correct and reasonable based on the above aspects? Answer your preferred hypothesis with #sat or #unsat. Let's think step by step.

Code: [[CODE]]
```

Fig. 3. Prompting template of the DCoL method. Prompts can be modified slightly according to specific tasks.

example, typically elicits a single line of reasoning without mechanisms for backtracking or exploring alternatives if that path proves incorrect. Given that for `SAT`/`UNSAT` problems, at least one of these two contrary reasoning paths (assuming `SAT` or assuming `UNSAT`) should lead to an accurate solution, a more effective approach is to explicitly guide LLMs to reason along both paths. By prompting the LLM to explore distinct reasoning processes for these contrary goals, it can generate more comprehensive analyses and ultimately produce more reliable answers with supporting explanations.

To this end, we propose a novel prompt method, DCoL. Fig. 2 illustrates a motivating example of our proposed DCoL approach. DCoL encourages LLMs to discard incorrect wrong prediction `UNSAT` after integrating dual paths of thinking, while COT only generates a wrong reasoning path, leading to the prediction error. As illustrated in the template of DCoL from Fig. 3, the prompt guides the LLM through a structured, multi-step reasoning process. The prompt first instructs the LLM to list and understand all variables and constraints added to the solver. It then poses the `SAT` hypothesis, asking the LLM to find satisfying variable assignments. Following this, it poses the contrary `UNSAT` hypothesis, instructing the LLM to find conflicting constraints. In the final step, the prompt directs the LLM to evaluate the analyses from both paths and determine which outcome, `SAT` or `UNSAT`, is more reasonable based on its findings. The logic code to be analyzed is represented by the placeholder `Code: [[CODE]]` at the bottom of the prompt, where the actual code would be inserted for evaluation.

DCoL, in contrast to COT, steers clear of falling into the erroneous reasoning trap illustrated in Fig. 2. Since y functions as an independent variable, it is inaccurate to interchangeably use $y = x + 1$ in the second constraint, as y in both constraints does not hold the same significance. This error, albeit subtle, often goes unnoticed even by humans lacking specialized knowledge. However, DCoL consistently provides a precise reasoning approach for obtaining accurate outcomes by thinking on the dual side of logic problems.

3.2. Agent-based DCoL implementation

While our prior work [17] demonstrated that the prompt-based implementation of DCoL is effective, it presents several limitations. First, the combined length of two parallel reasoning chains can exceed a model's maximum output tokens, leading to truncated and incomplete analyses. Second, tasking a single LLM with simultaneously handling two contradictory hypotheses can result in cognitive interference, where the reasoning paths influence and degrade one another.

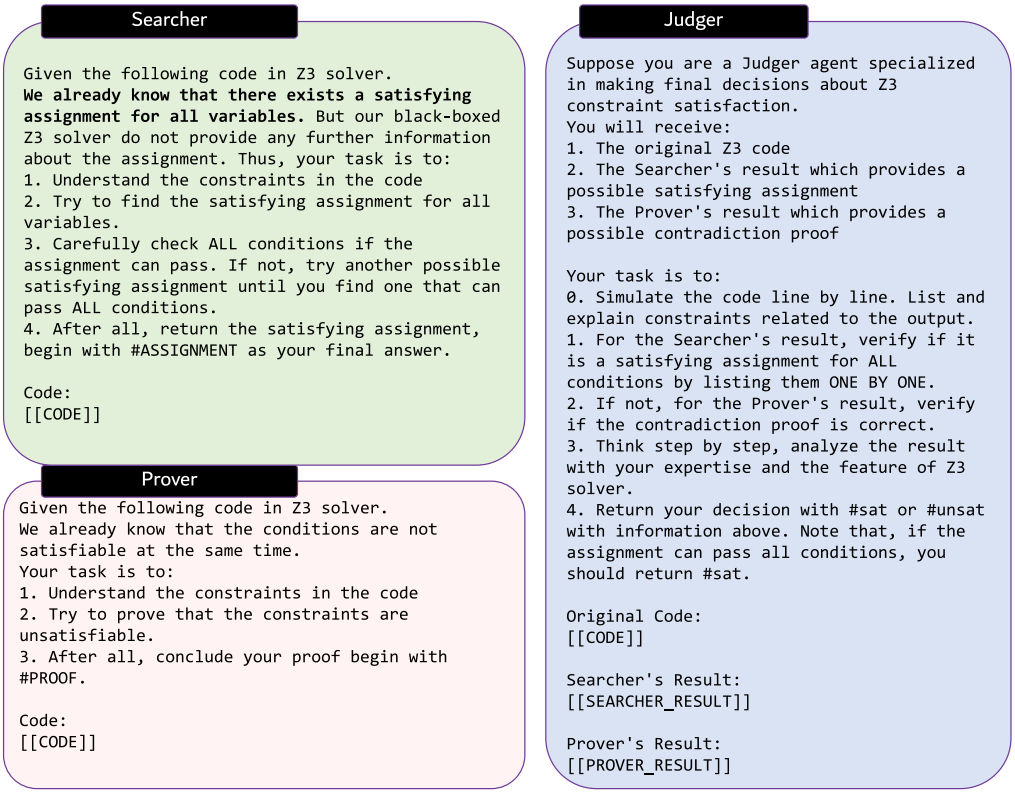


Fig. 4. Prompting template of the Agent-based DCoL method including a searcher, a prover, and a judger.

Third, this unified approach lacks a structured mechanism for error recovery; an initial mistake in one line of reasoning often compromises the final, integrated analysis.

To address these shortcomings, we evolved the prompt-based DCoL into a more robust, agent-based framework. Here, the concept of agents refers a set of distinct, role-based reasoning processes. Each process is specialized for a unique function and is initiated and guided by a specific prompt template, as shown in Fig. 4. This structure guide the LLM through a decoupled, multi-step reasoning process. In this upgraded architecture, the reasoning for the SAT and UNSAT hypotheses is decoupled and assigned to three distinct independent agents, as Fig. 4 illustrates.

- **Searcher**, responsible for searching for a valid variable assignment that satisfies all constraints. Its process involves iteratively modifying a proposed assignment while checking it sequentially against each constraint until a valid solution is found. To encourage a persistent search, the agent is primed with the assumption that a satisfying assignment exists.
- **Prover**, designed to prove the UNSAT hypothesis by identifying a fundamental logical contradiction or a minimal set of conflicting constraints. It is similarly primed with the opposing assumption, the constraints are inherently contradictory, to drive a rigorous proof-finding process.
- **Judger**, the final agent that receives and analyzes the complete outputs from both the Searcher and the Prover. A critical part of its role is to independently re-verify the assignment proposed by the Searcher against the problem's constraints. Finally, it assesses the quality and coherence of the two opposing arguments to render the final, most reasonable conclusion.

By splitting the reasoning process into specialized agents we effectively address the issues of cognitive interference and context limitations. This design reflects the fundamental difference between searching for a solution and verifying its correctness. Our primary objective is to use the semantic reasoning of large language models to intelligently locate a potential satisfying assignment. This allows the model to propose a specific candidate solution based on the logical intent of the code instead of performing an exhaustive search. This approach allows the framework to navigate logical constraints through pattern recognition which is often faster than the rigid formula by formula verification executed by traditional symbolic solvers because the neural model can directly analyze the logical essence of the code to propose plausible certificates. While this neural simulation approach does not provide the absolute precision or formal guarantees of symbolic tools we emphasize that the independent verification steps in both the Searcher and Judger agents serve as a critical safeguard against hallucinations. Consequently the agent based architecture ensures that the final result is supported by a double verified assignment for cases where a solution is found.

4. Implementation and setup

In this section, we present the details of our implementation and the setting of experiment, including the LLMs we investigated, the curation and details of the dataset, and baselines we selected.

4.1. Involved LLMs

The objective of this study is to assess the reliability and limitations of LLMs in simulating logic codes when compared to conventional solvers like Z3. To this end, we employ a diverse set of foundational models in our evaluation. This includes prominent open-source LLMs, such as the LLaMA family [8,27], alongside high-performance, closed-source models like the GPT family. We place special emphasis on recently emerged models engineered for advanced reasoning, such as Deepseek R1 [28] and Gemini [29]. Given their demonstrated ability to reason through complex mathematical and programming tasks, these models serve as highly relevant and powerful baselines for our investigation into logic code simulation. We present details of these models as follows:

GPT-3.5 Turbo: GPT-3.5 Turbo, also known as ChatGPT, is a decoder-only network based on the Transformer architecture, comprising 175 billion parameters. It has undergone pretraining on 45 terabytes of text data gathered from diverse sources like books, articles, and websites. Moreover, GPT-3.5 Turbo is optimized for invoking its capability with Reinforcement Learning from Human Feedback (RLHF) technique. The GPT-3.5 Turbo-0125 supports a context window of 16,385 tokens.

GPT-4 Turbo. GPT-4 Turbo is an advancement of GPT-4, offering increased capabilities with an updated knowledge cutoff extended to April 2023. It also features a 128k context window in a single prompt.

LLaMA-2. LLaMA-2, an open-source foundational model introduced by Meta, has been trained on 2 trillion tokens and is designed to accommodate a context length of 4096 by default. LLaMA-2 models have been refined with the input of more than 1 million human annotations, specifically tailored for chat purposes. This study focuses on the evaluation of the 13B-Chat model.

Code LLaMA. Code LLaMA is a refined version of LLaMA-2 to support various code-related activities like coding, testing, explaining, and finishing code segments. We also deploy CodeLLaMA-13B-Instruct for our evaluation.

Deepseek-R1. DeepSeek-R1 is a powerful open-source model distinguished by its advanced reasoning capabilities, which are primarily developed through reinforcement learning (RL) rather than traditional supervised fine-tuning. This model is designed to excel at tasks requiring complex logical steps, such as mathematics and coding, and has demonstrated performance comparable or superior to other leading models in these domains.

Gemini-2.5. Gemini 2.5 is a family of next-generation multimodal models from Google, featuring enhanced reasoning and a long context window. It is designed as a thinking model capable of reasoning through complex problems in code, math, and science before responding, which improves its accuracy and performance. Equipped with a 1-million token context window, it can analyze large datasets and codebases. This study utilizes the gemini-2.5-flash version.

4.2. Datasets

4.2.1. Functional and semantic datasets

To facilitate a deep analysis of how LLMs simulate realistic logical codes, we introduce three new datasets curated from various established sources.

- **Z3Tutorial**: We gathered 31 code examples from the official Programming Z3 tutorial [30] to form a foundational dataset that covers common Z3-Python functionalities.
- **Z3Test**: From the official Z3 repository, Z3test,[2] we extracted 85 Python test cases to create the *Z3Test* dataset. These instances are categorized into logic-only, arithmetic, and type-inference problems. For our primary experiments, we exclude type-inference questions (e.g., *is_add(x+y)*), reserving them as a separate challenge for logic code simulation.
- **SMTSim**: We curated 102 instances in SMT-LIB format from the SMT-COMP 2023 competition [31]. These problems are predominantly derived from industrial software and established problem collections, featuring complex theories such as real arithmetic (linear and non-linear), bit vectors, uninterpreted functions, and strings.

An overview of these datasets is presented in Table 1. It is important to note that the complexity of a logic problem is determined by the number and difficulty of its constraints, not just its lines of code. All snippets in these datasets have been filtered to ensure they yield a definitive `SAT` or `UNSAT` result. While public sources such as Z3Tutorial and Z3Test might have appeared in LLM training corpora, these sources contain only the logic code itself. This code represents the problem input. The corresponding `SAT/UNSAT` labels, which represent the problem solution, are not available in these public sources. These labels were generated by us via execution. Therefore, the task of predicting the outcome remains a valid test of reasoning, not memorization.

[2] https://github.com/Z3Prover/z3test

Table 1
Dataset information provided. Format means the input format of questions. Z3Py is based on Python, while SMTLIB is the standard language of SMT solvers. Mean LOC represents the average Lines of Code (LoC) of examples.

Dataset	Formulation	Format	# of samples	Mean LoC
ProntoQA	PL	NL	500	–
Z3Tutorial	SMT	Z3Py	31	9.90
Z3Test	SMT	Z3Py	85	8.37
SMTSIM	SMT	SMTLIB	104	14.36
3SATSIM	3-SAT	Generated	200	58.17

4.2.2. Complexity and scalability dataset

The datasets described above are designed to test the functional and semantic aspects of solver simulation, such as grammar, behavior, and data type support. However, they do not systematically investigate scalability or the ability to solve computationally hard problems. To address this, we introduce the **3SATSIM** dataset, which focuses on 3-SAT problems of increasing complexity. While randomly generated 3-SAT problems are a standard benchmark for solver performance [32,33], our contribution is the creation of a new dataset where these problems are translated into executable Z3Py format. This dataset is specifically designed to evaluate the scalability of LLMs on our task of logic code simulation. Problems were randomly generated with a moderate clause-to-variable ratio between 5.6 and 5.8. As a higher ratio tends to produce over-constrained (`UNSAT`) problems, while a lower ratio results in under-constrained (`SAT`) ones [34–36].

In practice, we created four difficulty levels by varying the number of variables (5, 8, 10, and 12). Each level contains 50 samples, equally balanced between `SAT` and `UNSAT` instances, resulting in a total of 200 examples. All examples are translated into executable Z3py format. From the statistical results presented in Table 1, we can infer that the 3SATSIM is the most challenging dataset as it has the longest length of code.

4.3. Prompting baselines

The baseline Prompting methods considered in this work are listed below:

- Standard Prompting (SD) [7]. SD directly asks LLMs to predict the execution results of logic codes. The LLMs will return only `SAT/UNSAT`, sometimes with short explanations.
- Chain-of-Thought (COT) [37]. COT encourages LLMs to generate a systematic solution by explaining each step before presenting the final answer. The LLMs will return a reasoning path tackling the issue raised.
- Plan-and-Solve Prompting (PS) [38]. PS instructs LLMs to devise a detailed plan for addressing issues and solve plan targets sequentially.
- Chain of Simulation (CoSm) [26]. The CoSm method enhances the conventional Chain of Thought prompting technique by circumventing the drawbacks associated with memorization in code simulation tasks. It aims to track the program trace and forces LLMs to simulate each instruction sequentially.

Additionally, Self-Consistency (SC) [39] utilizes a majority voting mechanism across different reasoning paths to enhance prediction robustness. The key insight behind this method is ensuring consistency

Table 2
Performance of LLMs with different prompt techniques. The **bolded** numbers denote the best performance in this dataset, while results in grey boxes represent the best result among all the prompt methods. All experiments are conducted three times.

Method	Z3Tutorial				Z3Test		
		Accuracy	Unknown	Exe. Acc.	Accuracy	Unknown	Exe. Acc.
Z3		100.0	–	100.0	98.80	1.20	100.0
GPT-4 Turbo	SD	84.95	–	84.95	76.47	3.53	79.27
	PS	86.02	–	86.02	82.35	–	82.35
	CoSm	81.72	–	81.72	80.39	2.35	82.32
	COT	86.02	–	86.02	81.18	2.35	83.13
	DCoL	86.02	–	86.02	83.53	1.76	84.52
LLaMA 2-13B	SD	38.70	19.35	48.00	40.00	8.23	43.03
	PS	54.83	9.67	60.71	42.35	8.23	46.15
	CoSm	61.29	9.67	67.85	34.11	2.35	34.93
	COT	53.22	8.06	57.89	50.00	5.88	52.79
	DCoL	64.51	6.45	68.96	54.11	1.17	54.76
Code LLaMA 13B	SD	58.06	3.22	60.00	57.64	5.88	61.25
	PS	51.61	12.90	59.25	55.29	10.58	61.84
	CoSm	70.96	–	70.96	61.17	9.41	67.53
	COT	56.45	17.74	68.62	58.23	3.52	60.36
	DCoL	70.96	6.45	75.86	62.35	–	62.35
Gemini 2.5	SD	86.02	–	86.02	93.72	–	93.72
	PS	87.10	–	87.10	93.33	–	93.33
	CoSm	84.95	–	84.95	96.07	–	96.07
	COT	87.10	–	87.10	96.47	–	96.47
	DCoL	89.25	–	89.25	96.07	–	96.07
	DCoL-A	**96.78**	–	**96.78**	96.47	–	96.47
Deepseek R1	SD	87.10	–	87.10	94.5	–	94.5
	PS	89.24	–	89.24	95.29	–	95.29
	CoSm	89.24	–	89.24	96.07	–	96.07
	COT	91.39	–	91.39	**96.86**	–	**96.86**
	DCoL	92.47	–	92.47	95.68	–	95.68
	DCoL-A	92.47	–	92.47	**96.86**	–	**96.86**

in the reasoning process. Since SC operates as a parallel prompting method, it can be combined with other prompting approaches, including ours. Therefore, we evaluate it as a standalone plugin.

Note that our proposed task is a novel one focusing on codes as initial input. Thus, methods for processing natural language problems [9,40,41], are no longer compatible with our framework. Furthermore, these approaches are limited in their ability to address a specific portion of the task we have outlined, such as SAT [41] and SMT-Int (SMT questions with integer computing theory) [9,40]. As a result, we do not include them as baselines in our study.

A notable observation is that the typical few-shot prompt, which is also known as in-context learning is ineffective in our experimental setting, which is also reported in similar tasks such as code simulation [26] and discussed in several works [42]. Few-shot prompt learning may not work on a task whose input-label correspondence is not already captured in the LLMs. In our logic code simulation task, elementary operations in different subtasks such as different function calls and various theories can be challenging for in-context learning. Besides, considering the intricate nature of logic problems, the templates heavily exceed the LLMs' context limit. Thus, we focus on the zero-shot prompting scheme in this research.

5. Evaluation

5.1. Functional and semantic evaluation

We first investigate whether LLMs can functionally and semantically solve logic problems by evaluating them on the Z3Tutorial and Z3Test dataset.

5.1.1. Effectiveness

We employ three LLMs from the conference version [17], GPT-3.5-Turbo, GPT-4-Turbo, LLaMA-2-13B, and CodeLLaMA-13B, with different prompting methods, to explore their capability of them on simulating logic codes. The results are provided in Table 2. Two datasets based on Z3Py are utilized for a comprehensive analysis. The *DCoL-A* represents the agentic version of DCoL method.

We utilize two additional metrics, namely "unknown rate" and "execution accuracy" to further investigate the effectiveness of logic code simulation.

Unknown rate indicates the percentage of instances such as 'If a solution exists, the output will be `SAT`; otherwise `UNSAT`', where LLMs struggle to provide a definitive answer. An unknown output can be caused by errors in the program, such as 'The code should output ERROR rather than `SAT` or `UNSAT`'.

Execution accuracy refers to the precision among instances that LLMs confirm to be `SAT`/`UNSAT`. This measure aids in assessing the reliability of the outputs generated by LLMs. This metric can be computed as $EA = \frac{Acc}{1-UNK}$.

A model can have low accuracy while obtaining a high execution accuracy. If we apply a random guessing procedure on those unknown examples, the accuracy will be acceptable. Note that all methods obtain execution accuracy greater than 50%, which means LLMs can perform at least stronger than random guessing.

The main observations are summarized below:

LLMs can pretend logic solvers. As shown in Table 2, GPT shows its capability to simulate logic codes even with the standard prompting method (SD). In the Z3Tutorial dataset analysis, it is evident that employing standard prompts can yield an accuracy exceeding 80% when utilizing GPT-4 Turbo, known for its high resilience among LLM options. Even when employing SD prompts with GPT-3.5, an accuracy of 67.74% can be attained. As for the Z3Test dataset, GPT-3.5 achieves over 70% accuracy, while GPT-4 successfully solved 80% logic problems in this dataset. This phenomenon comes from the combination of code understanding and logical reasoning for GPT models. Moreover, among all LLMs tested, the DCoL consistently enhances performance in logic code simulation across all datasets, with gains of up to 7.06% on GPT-4.

Superior Performance of Reasoning-Enhanced LLMs. Models engineered for reasoning, such as Gemini and Deepseek, significantly

Input

```
p, q = Bools('p q')

demorgan =
And(p, q) == Not(Or(Not(p), Not(q)))
```

Output

1. The constraint states that the logical AND of p and q should be equal to the logical NOT of the OR of the negations of p and q. This is a fundamental property of boolean logic.

2. We need to consider all possible assignments.

3. Since the constraint is based on De Morgan's law, it is reasonable to assume that there are no conflicts in the constraint. Therefore, the output of the code should be SAT.

Fig. 5. An example of De Morgan's Law.

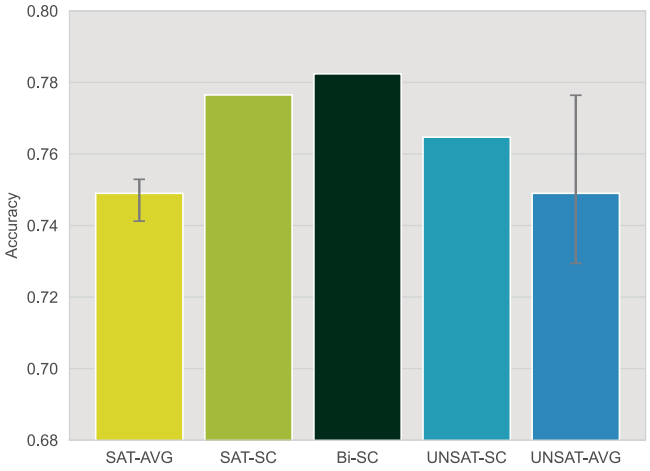


Fig. 6. The accuracy results of different order of SAT and UNSAT in the prompt.

outperform the other LLMs evaluated. Benefiting from reinforcement learning on extensive code and math corpora, these models leverage sophisticated internal reasoning mechanisms to deconstruct and solve complex logic puzzles. This superiority is evident in our results (Table 2), where both Gemini and Deepseek consistently achieve accuracies exceeding 90% across most configurations on both the Z3Tutorial and Z3Test datasets. Notably, when paired with our DCoL and agent-based DCoL-A methods, their performance often surpasses 95%, demonstrating a powerful synergy between advanced model architecture and structured reasoning frameworks.

Think on logic rather than think on code. A representative method performing *Think on Code* is Chain of Simulation (CoSm), which forces LLMs to simulate the program instruction by instruction. However, this method is only suitable for simulating problems with sequential execution paths. Logic reasoning is a more complex issue as the execution chain of logic codes is hidden under external solver libraries such as the Conflict Driven Clause Learning algorithm applied in Z3 solver. To solve logic problems, LLMs should pay more attention to logical reasoning to derive the correct execution path. Nevertheless, the execution of code should not be ignored in our task. A typical error case involves adding constraints to a solver that are never actually verified, where LLMs should disregard these constraints.

LLMs exceed some theoretical limitations of solvers A motivated example of this strength is showcased in Fig. 1. Constraints like $2^x == 3$ go beyond the representation capability of SMT theories. Although it remains such a simple question for humans, typical solvers like Z3 output UNKNOWN in this case.

LLMs leverage explainable knowledge for reasoning. We provide an interesting example in Fig. 5 targeting checking the satisfiability of De Morgan's Law. Despite enumerating all assignments for proving, LLMs build the connection from logic codes to De Morgan's Law and make the correct prediction with this external knowledge. We suggest that leveraging formulas and theorems is a good manual for complex problem-solving, and LLMs show their capability of utilizing knowledge for more explainable reasoning.

DCoL prompt is effective. Our proposed prompt method, DCoL, offers a structured approach to LLM reasoning by encouraging two separate lines of problem-solving, and the results in Table 2 effectively support its efficacy. Across different datasets, particularly with the Z3Tutorial and Z3Test, we see that DCoL consistently outperforms other methods or shows competitive results, especially in the execution accuracy (Exe. Acc.) and accuracy rates, where it often achieves the best or near-best performance, as highlighted in gray. For instance, GPT-4 Turbo achieves 86.02% accuracy using DCoL in the Z3Tutorial dataset, matching other top-performing methods while outperforming others in execution accuracy on the Z3Test dataset with 83.53%. This consistent performance boost suggests that the dual-reasoning structure introduced by DCoL is highly compatible with the evolving capabilities of LLMs. It facilitates multi-way reasoning, allowing the model to explore multiple solution paths, which can reduce the UNK output rate, indicating fewer situations where the model fails to provide meaningful answers. While the GPT models may occasionally follow a single reasoning pathway, as seen in experiments, the results show that DCoL still effectively improves overall model accuracy and reduces failure cases, confirming its utility in logic code simulation and problem-solving.

Bi-directional Self-Consistency improves performance. Prior work suggests that the order of premises can significantly impact an LLM's reasoning performance in natural language tasks [43]. Our DCoL prompt, which sequentially queries the model on SAT and UNSAT hypotheses, is susceptible to this ordering effect; we observed that swapping the order of the reasoning paths could alter the model's final prediction. To mitigate this bias and improve robustness, we designed a validation experiment inspired by the principle of Self-Consistency. The experiment involved sending both SAT-first and UNSAT-first DCoL prompts to GPT-3.5-Turbo three times each for the Z3Test dataset. The final answer was then determined by a majority vote over the aggregated responses. As depicted in Fig. 6, this Bi-directional Self-Consistency (Bi-SC) approach yielded a 3.34% improvement in average performance, effectively bridging the gap caused by the differing prompt orders.

Based on the experimental results, we believe that LLMs, particularly the GPT family, **serve as highly effective simulators**, especially when utilizing our proposed DCoL prompt and Bi-directional Self-Consistency mechanism.

5.1.2. Robustness

We have already demonstrated the capability of LLMs in code simulation, indicating that LLMs can comprehend our intentions based on the provided code. Since LLMs can predict the outcome by analyzing the code's logic rather than invoking the underlying external solver, they are more tolerant of syntax errors compared to traditional solvers with parsers. To assess the robustness of code simulation in LLMs, we conducted an experiment involving four distinct syntax error patterns:

- Mismatched Parentheses: there is an inconsistency in the placement of parentheses, such as an unequal number of opening and closing parentheses or improper nesting, such as $(a + b$ or $func(a, b))$.

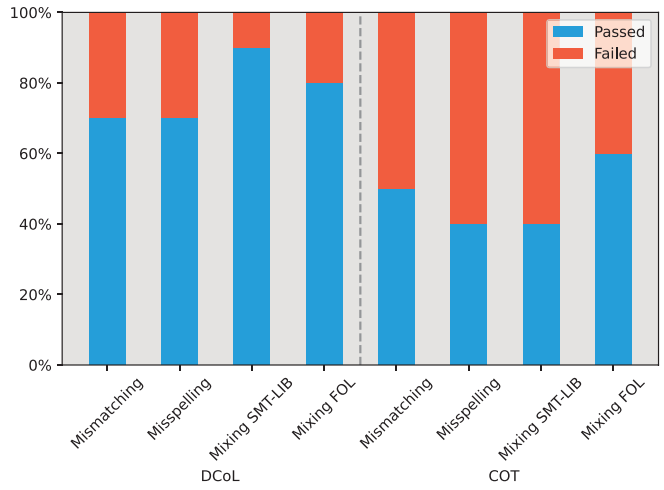


Fig. 7. Performance drop against code-level modifications.

Table 3
ProntoQA comparison.

Method	Accuracy
Direct Solving	
SD	51.20
COT	72.00
COT + CodeGen	76.80
Translation + Solver	
Logic-LM	61.00
Translation + Simulation	
Code Only	50.20
NL + Code	75.80

- Misspelled Variable or Function Names: incorrect spelling or naming of variables or functions within the code, such as $x1$ and xl.
- Mixing Z3 Library Grammar: elements specific to the SMT-LIB, such as $(declare-fun\ \langle symbol\rangle\langle sort\rangle^*)$, are incorrectly integrated into the code syntax.
- Mixing First-Order Logic Grammar: elements of first-order logic grammar, such as quantifiers ∀ and ∃, are incorrectly incorporated into the code syntax.

For each syntax error pattern, we randomly selected 10 cases from the Z3 Test dataset to introduce errors. Subsequently, we applied both COT and our proposed method (DCoT) on the poisoned Z3 Test dataset. As illustrated in Fig. 7, the code simulation of LLMs with COT was significantly misled by the introduced syntax errors, leading to a decrease in accuracy of approximately 40%. In contrast, syntax errors had minimal influence on the effectiveness of our method, which demonstrates **LLMs are robust simulators**.

LLM can simulate generated logic codes. This section is motivated by the limitations of Logic-LM [9], which tackles logic inference in natural language through a translation-solver approach. While Logic-LM achieves almost 100% success in generating symbolic codes for the ProntoQA dataset, its accuracy sharply drops to 61% when interacting with symbolic solvers. This decline is largely due to **LLMs generating unreliable or implicit codes**, where hidden knowledge inferred by the LLM is not explicitly present in the generated code. For instance, an LLM might generate a code segment that assumes a certain variable should be non-negative based on common sense reasoning, even though this constraint is not included in the code itself. Consequently, a traditional symbolic solver, unaware of this implicit assumption, may infer an incorrect answer. To mitigate this issue, we propose ensuring

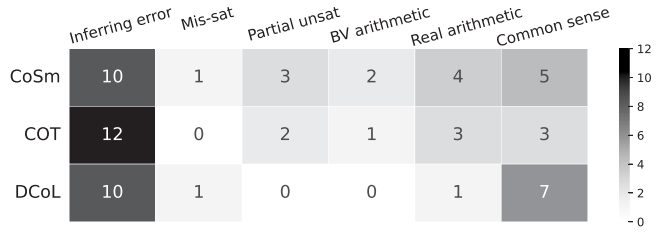


Fig. 8. Error analysis among different methods.

consistency between code generation and the solver's understanding by simulating logic on generated outputs. As shown in Table 3, we conducted a comparative study based on Logic-LM's shortcomings. The *COT + CodeGen* approach treats code generation as an auxiliary in-context task, while *NL + Code* provides the original language input along with previously generated code. Our results indicate that even with low-quality code, direct simulation performs comparably to standard prompting for natural language tasks, with *NL + Code* achieving an accuracy of 75.80%, which is comparable to *COT + CodeGen* and outperforms the Logic-LM baseline that relies on external solvers.

5.1.3. Error analysis

We analyzed results from the Z3Test dataset using three prompting methods: CoSm, COT, and DCoL, aiming to identify error types across different reasoning paths. The errors were categorized into six types as Fig. 8 illustrates,

Inference Error: Mistakes in logical deduction, such as misjudging $a \wedge b$ as false when both a and b are `True`, or conflicting conclusions like first inferring $a > 10$ and then $a < 10$.

Misunderstanding Satisfiability (SAT) Error: Occurs when an LLM correctly identifies a solution set for a satisfiable (SAT) problem but wrongly classifies it as UNSATisfiable (`UNSAT`), due to misinterpreting the problem as having multiple solutions.

Partial `UNSAT` Error: Arises when the ground truth is satisfiable, but the LLM finds conflicting assignments while ignoring valid ones, wrongly classifying the problem as `UNSAT`. For example, $x = 1, y = 0$ is satisfiable, but the LLM may mistakenly label $x = 1, y = 2$ as an `UNSAT` core.

Bit-vector Arithmetic Error: Involves errors in handling bitwise operations on bit vectors, like incorrectly interpreting a left shift operation as an overflow.

Real Arithmetic Error: Includes computational mistakes with real numbers, such as confusion over the equality of positive and negative zeros in floating-point numbers.

Commonsense Error: Basic mathematical or logical mistakes, such as equating 1 with 2 or failing to recognize common library functions, indicating gaps in the LLM's understanding.

Our analysis shows that DCoL improves LLMs' understanding of satisfiability problems, reducing misinterpretations and incorrect judgments. In arithmetic tasks, DCoL enhances performance by guiding the LLM through initial steps.

While not the focus of our study, certain industry examples are noteworthy. For instance, the `is_add()` function in Z3Py, which checks if an expression is an addition, can lead to confusion; `is_add(1 + 2)` may return false as Z3Py treats 1 + 2 as a constant, potentially confusing LLMs and users unfamiliar with this behavior.

Additionally, in the *SMTSim* dataset from SMT-COMP 2023, some SMT files generated by complex backend libraries contain numerous variables and formulas. Due to LLM token limits, we filtered out larger examples, yet many of these filtered cases remain unsolved by solvers like Z3, suggesting potential areas for future research.

Table 4
SMTSim results.

Method		ACC.	UNK.	Exe. Acc
Z3 Solver		97.06	2.94	100.0
GPT-3.5 Turbo	COT	5.88	85.29	33.33
	CoSm	14.71	78.43	68.20
	DCoL	**54.9**	6.86	58.94
GPT-4 Turbo	COT	51.96	19.61	65.92
	CoSm	50.98	23.53	66.67
	DCoL	**58.82**	5.82	62.45

Table 5
Performance of different LLMs and prompting methods on the 3SATSIM dataset, showing accuracy across different data splits.

Method		SAT	UNSAT	Overall
GPT-4o	SD	15.0	80.0	47.5
	COT	15.0	81.0	48.0
	DCoL	39.0	66.0	52.5
	DCoL-A	26.0	84.0	55.0
Gemini-2.5	SD	21.0	77.0	49.0
	COT	22.0	79.0	50.5
	DCoL	34.0	85.0	59.5
	DCoL-A	52.0	95.0	73.5
Deepseek-R1	SD	37.0	81.0	59.0
	COT	40.0	83.0	61.5
	DCoL	58.0	90.0	74.0
	DCoL-A	82.0	95.0	**88.5**

5.1.4. Hard problems

SMTSim dataset contains complicated logic code, with more lines of codes, variables, and theories. We regard it as a challenge for logic code simulation. We report the performance of GPT families in Table 4.

We observe that COT and CoSm suffer from UNKNOWN prediction, while our proposed DCoL achieves over 50% accuracy even with GPT-3.5-Turbo. However, the relatively low execution accuracy reveals that LLMs struggle to solve such complex datasets. This issue may stem from the abundance of variables and constraints present. In the SMTSim dataset, numerous instances occur infrequently but are deeply intertwined. Due to their limitations in handling extensive contexts, LLMs struggle to address complex logical compositions effectively.

5.2. Complexity and scalability evaluation

In this section, we focus on leveraging the most challenging dataset, 3SATSIM, to evaluate LLMs performance on complex 3-SAT problems, and scalability on them.

5.2.1. Overall performance and model hierarchy

The 3SATSIM dataset proves to be exceptionally challenging as shown in Table 5, establishing a clear performance hierarchy among the models: Deepseek-R1 demonstrates a commanding lead, followed by Gemini-2.5, and then GPT-4o. GPT-4o exhibited poor instruction-following capabilities; it often tended to guess the final result without providing the detailed variable assignments or `UNSAT` cores required by the prompt, limiting its practical utility. Gemini-2.5 performed better but showed a specific weakness in solving for satisfying assignments, frequently proposing incorrect solutions for `SAT` instances. Deepseek-R1's superior accuracy achieving **88.5%** with DCoL-A, suggests it is more adept at both the complex reasoning and the instruction-following aspects of the task.

5.2.2. UNSAT performance advantage

Gemini said We have observed that all models perform significantly better on the UNSAT split than on the SAT split. This discrepancy arises from the logical distinction between identifying a precise certificate for satisfiability and the more straightforward process of verifying the inconsistency of proposed assignments. Our agentic framework utilizes the fact that a targeted verification path through the Judger agent is more reliable than an open ended search conducted by the Searcher agent. When the Searcher agent fails to find a valid assignment and proposes an incorrect one the Judger agent can effectively detect its invalidity through localized reasoning. This successful verification of a faulty certificate often leads the Judger agent to correctly determine that the problem is UNSAT. Consequently the accuracy on the UNSAT split is consistently higher as shown by the high performance levels achieved with DCoL A. This architectural design results in an accuracy of 95.0 percent for Gemini 2.5 and 94.0 percent for DeepSeek R1 on the UNSAT instances. This process also reveals a specific failure mode when solving SAT instances where finding the correct assignment remains a significant bottleneck. Further validation on more diverse and complex datasets is necessary to fully assess the models genuine proof capabilities for unsatisfiability

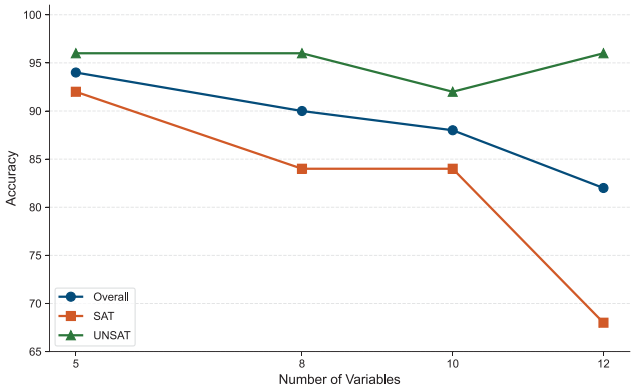


Fig. 9. Performance of the Deepseek-R1 model with the DCoL-A framework on the 3SATSIM dataset, broken down by problem size (number of variables). The plot illustrates the divergence in accuracy between `SAT` and `UNSAT` problem splits as complexity increases.

5.2.3. Efficacy of agent-based DCoL

The results highlight the significant advantage of the agent-based framework, textbfDCoL-A over the prompt-based **DCoL**. For Gemini-2.5, the overall accuracy jumps from 59.5% with DCoL to 73.5% with DCoL-A. Similarly, GPT-4o improves from 52.5% to 55.0%. While Deepseek-R1 reaches the top among all baselines with 88.5%. This consistent improvement demonstrates that the structured, multi-agent architecture is critical for tackling these hard problems. The `Judger`'s independent verification role is key; it not only catches errors from the `Searcher` but also compensates for the inherent weaknesses of the models, such as Gemini-2.5's tendency to generate faulty solutions. This robust, double-check mechanism is the primary reason for DCoL-A's superior performance compared to the single-pass DCoL prompt.

5.2.4. Scalability analysis

Fig. 9 provides a visual representation of the model's performance on the 3SATSIM dataset, revealing how accuracy scales with problem complexity. The analysis of this plot highlights three critical observations.

First, there is a clear downward trend in the overall accuracy as the number of variables increases, falling from 94.0% on the simplest problems (5 variables) to 82.0% on the most complex (12 variables). This trend visually confirms that, despite the advanced agentic framework, the model's ability to solve 3-SAT problems degrades as the search space expands, underscoring the inherent challenge of scalability.

Second, the plot starkly illustrates the performance asymmetry between solving `SAT` and `UNSAT` instances. The accuracy on the `UNSAT`

split remains exceptionally high and stable, showing little degradation with increased complexity. In sharp contrast, the accuracy on the SAT split shows a steep and consistent decline, dropping from 92.0% down to 68.0%. This divergence provides strong evidence that the model excels at the deductive task of identifying contradictions but struggles with the constructive task of finding a valid assignment in a larger search space.

Finally, the stability of the UNSAT performance, combined with the decline in SAT performance, suggests that the DCoL-A framework's Judger is effective at identifying faulty assignments from the Searcher. However, it also reveals that the Searcher's core ability to find a solution is the primary bottleneck. This performance asymmetry and the steep decline in SAT accuracy connect directly to complexity theory. Finding a satisfying assignment is a constructive task that requires navigating a combinatorial search space that grows exponentially with the number of variables. This empirical result highlights a known architectural limitation of attention-based models, which excel at learned heuristics and pattern recognition but struggle with the systematic, bounded search required by NP-complete problems. This indicates that while the agentic framework significantly improves robustness, the fundamental challenge for LLMs in scalable combinatorial problem-solving remains an open area for future work.

6. Conclusion

In this work, we introduce the novel concept of logic code simulation to evaluate the ability to comprehend, analyze, and simulate logic problems encoded in programs. We propose DCoL, a novel prompt fashion designed to enhance reasoning performance on code-based logic problems. Furthermore, we release three datasets collected from the solver community and perform an extensible assessment to evaluate the capability of logic code simulation for various LLMs. In the conclusion, the authors mention a potential avenue for future work involving the application of the DCoL technique to convert natural language into logical code. Apriori, it is not immediately clear how this can be achieved, but exploring this direction could provide valuable insights.

One possible approach involves leveraging the dual reasoning paths central to DCoL, where the first path focuses on interpreting natural language inputs and extracting the logical structure or constraints embedded in the text. The second path could then focus on translating this understanding into executable logical code, ensuring consistency between the extracted logic and the generated code. This dual-path system could help bridge the gap between natural language reasoning and formal logic, addressing the ambiguity that often arises when translating nuanced human language into strict logical frameworks. Additionally, integrating techniques such as knowledge retrieval and structured prompting could allow LLMs to retrieve relevant logical patterns from a knowledge base while maintaining flexibility in the translation process.

This integration would represent a significant step forward in developing LLM-based solvers capable of handling not only traditional logic problems but also more complex, real-world scenarios where logical rules are derived from natural language descriptions. By enhancing DCoL in this way, future research could establish a new paradigm in logic-based problem-solving, where natural language inputs are seamlessly converted into formal logic programs. Another promising avenue for future work is the development of hybrid approaches. Given that DCoL-A struggles with the constructive search for SAT assignments, it could be integrated with traditional tools. For example, the LLM could act as a powerful heuristic guide to prune the search space for a conventional combinatorial search algorithm, combining the LLM's pattern recognition with the solver's rigorous search capabilities. Furthermore, we plan to broaden DCoL's application beyond traditional solvers like SAT or UNSAT, ensuring its effectiveness across diverse logic code scenarios and complex logic programs, paving the way for more efficient LLM-driven logic solvers.

CRediT authorship contribution statement

Minyu Chen: Writing – original draft, Software, Methodology, Formal analysis. **Ling-I Wu:** Data curation. **Ruibang Liu:** Software, Data curation. **Xi Chang:** Resources. **Jianxin Xue:** Resources. **Guoqiang Li:** Supervision, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Guoqiang Li reports financial support was provided by Shanghai Jiao Tong University. Guoqiang Li reports a relationship with Shanghai Jiao Tong University that includes: employment. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work is supported by the Key Program of the National Natural Science Foundation of China with Grant No. 62532014.

References

[1] L. De Moura, N. Bjørner, Z3: An efficient SMT solver, in: International Conference on Tools and Algorithms for the Construction and Analysis of Systems, Springer, 2008, pp. 337–340.
[2] H. Barbosa, C. Barrett, M. Brain, G. Kremer, H. Lachnitt, M. Mann, A. Mohamed, M. Mohamed, A. Niemetz, A. Nötzli, et al., cvc5: A versatile and industrial-strength SMT solver, in: International Conference on Tools and Algorithms for the Construction and Analysis of Systems, Springer, 2022, pp. 415–442.
[3] L. Cordeiro, B. Fischer, Verifying multi-threaded software using SMT-based context-bounded model checking, in: Proceedings of the 33rd International Conference on Software Engineering, 2011, pp. 331–340.
[4] G. Soltana, M. Sabetzadeh, L.C. Briand, Practical constraint solving for generating system test data, ACM Trans. Softw. Eng. Methodol. (TOSEM) 29 (2) (2020) 1–48.
[5] E. Kang, S. Lafortune, S. Tripakis, Automated synthesis of secure platform mappings, in: Computer Aided Verification: 31st International Conference, CAV 2019, New York City, NY, USA, July 15-18, 2019, Proceedings, Part I 31, Springer, 2019, pp. 219–237.
[6] M.R. Gadelha, E. Steffinlongo, L.C. Cordeiro, B. Fischer, D. Nicole, SMT-based refutation of spurious bug reports in the clang static analyzer, in: 2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion), IEEE, 2019, pp. 11–14.
[7] T. Brown, B. Mann, N. Ryder, M. Subbiah, J.D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., Language models are few-shot learners, Adv. Neural Inf. Process. Syst. 33 (2020) 1877–1901.
[8] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, et al., Llama 2: Open foundation and fine-tuned chat models, 2023, arXiv preprint arXiv:2307.09288.
[9] L. Pan, A. Albalak, X. Wang, W. Wang, Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning, in: Findings of the Association for Computational Linguistics, EMNLP 2023, 2023, pp. 3806–3824.
[10] J. Lee, W. Hwang, SymBa: Symbolic backward chaining for multi-step natural language reasoning, 2024, arXiv preprint arXiv:2402.12806.
[11] J. Feng, R. Xu, J. Hao, H. Sharma, Y. Shen, D. Zhao, W. Chen, Language models can be logical solvers, 2023, arXiv preprint arXiv:2311.06158.
[12] O. Tafjord, B. Dalvi, P. Clark, ProofWriter: Generating implications, proofs, and abductive statements over natural language, in: Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021, 2021, pp. 3621–3634.
[13] A. Saparov, H. He, Language models are greedy reasoners: A systematic formal analysis of chain-of-thought, in: The Eleventh International Conference on Learning Representations, 2023, URL: https://openreview.net/forum?id=qFVVBzXxR2V.
[14] S. Han, H. Schoelkopf, Y. Zhao, Z. Qi, M. Riddell, L. Benson, L. Sun, E. Zubova, Y. Qiao, M. Burtell, et al., Folio: Natural language reasoning with first-order logic, 2022, arXiv preprint arXiv:2209.00840.
[15] W. Zhong, S. Wang, D. Tang, Z. Xu, D. Guo, Y. Chen, J. Wang, J. Yin, M. Zhou, N. Duan, Analytical reasoning of text, in: Findings of the Association for Computational Linguistics: NAACL 2022, 2022, pp. 2306–2319.
[16] Z. Li, Y. Cao, X. Xu, J. Jiang, X. Liu, Y.S. Teo, S.-w. Lin, Y. Liu, LLMs for relational reasoning: How far are we? 2024, arXiv preprint arXiv:2401.09042.

[17] M. Chen, G. Li, L.-I. Wu, R. Liu, Y. Su, X. Chang, J. Xue, Can language models pretend solvers? logic code simulation with llms, in: International Symposium on Dependable Software Engineering: Theories, Tools, and Applications, Springer, 2024, pp. 102–121.

[18] J. Pérez, P. Barceló, J. Marinkovic, Attention is turing-complete, J. Mach. Learn. Res. 22 (75) (2021) 1–35.

[19] C. Wei, Y. Chen, T. Ma, Statistically meaningful approximation: a case study on approximating turing machines with transformers, Adv. Neural Inf. Process. Syst. 35 (2022) 12071–12083.

[20] A. Giannou, S. Rajput, J.-y. Sohn, K. Lee, J.D. Lee, D. Papailiopoulos, Looped transformers as programmable computers, in: International Conference on Machine Learning, PMLR, 2023, pp. 11398–11442.

[21] D. Schuurmans, Memory augmented large language models are computationally universal, 2023, arXiv preprint arXiv:2301.04589.

[22] G. Kim, P. Baldi, S. McAleer, Language models can solve computer tasks, Adv. Neural Inf. Process. Syst. 36 (2024).

[23] A.Z. Yang, C. Le Goues, R. Martins, V. Hellendoorn, Large language models for test-free fault localization, in: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, 2024, pp. 1–12.

[24] C. Liu, S. Lu, W. Chen, D. Jiang, A. Svyatkovskiy, S. Fu, N. Sundaresan, N. Duan, Code execution with pre-trained language models, in: Findings of the Association for Computational Linguistics: ACL 2023, 2023, pp. 4984–4999.

[25] Z. Shi, M. Li, Y. Liu, S. Khan, J. Huang, H.-L. Zhen, M. Yuan, Q. Xu, SATformer: Transformer-based UNSAT core learning, in: 2023 IEEE/ACM International Conference on Computer Aided Design, ICCAD, IEEE, 2023, pp. 1–4.

[26] E. La Malfa, C. Weinhuber, O. Torre, F. Lin, A. Cohn, N. Shadbolt, M. Wooldridge, Code simulation challenges for large language models, 2024, arXiv preprint arXiv:2401.09074.

[27] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X.E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin, et al., Code llama: Open foundation models for code, 2023, arXiv preprint arXiv:2308.12950.

[28] DeepSeek-AI, D. Guo, D. Yang, H. Zhang, J.-M. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, X. Zhang, X. Yu, Y. Wu, Z.F. Wu, Z. Gou, Z. Shao, Z. Li, Z. Gao, A. Liu, B. Xue, B.-L. Wang, B. Wu, B. Feng, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Chen, D.-L. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Ding, H. Xin, H. Gao, H. Qu, H. Li, J. Guo, J. Li, J. Wang, J. Chen, J. Yuan, J. Qiu, J. Li, J. Cai, J. Ni, J. Liang, J. Chen, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Zhao, L. Wang, L. Zhang, L. Xu, L. Xia, M. Zhang, M. Zhang, M. Tang, M. Li, M. Wang, M. Li, N. Tian, P. Huang, P. Zhang, Q. Wang, Q. Chen, Q. Du, R. Ge, R. Zhang, R. Pan, R. Wang, R.J. Chen, R. Jin, R. Chen, S. Lu, S. Zhou, S. Chen, S. Ye, S. Wang, S. Yu, S. Zhou, S. Pan, S.S. Li, S. Zhou, S.-K. Wu, T. Yun, T. Pei, T. Sun, T. Wang, W. Zeng, W. Zhao, W. Liu, W. Liang, W. Gao, W.-X. Yu, W. Zhang, W. Xiao, W. An, X. Liu, X. Wang, X. Chen, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yang, X. Li, X. Su, X. Lin, X.Q. Li, X. Jin, X.-C. Shen, X. Chen, X. Sun, X. Wang, X. Song, X. Zhou, X. Wang, X. Shan, Y.K. Li, Y.Q. Wang, Y.X. Wei, Y. Zhang, Y. Xu, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Yu, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Ou, Y. Wang, Y. Gong, Y.-J. Zou, Y. He, Y. Xiong, Y.-W. Luo, Y. mei You, Y. Liu, Y. Zhou, Y.X. Zhu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Y. Tang, Y. Zha, Y. Yan, Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Xie, Z. guo Zhang, Z. Hao, Z. Ma, Z. Yan, Z. Wu, Z. Gu, Z. Zhu, Z. Liu, Z.-A. Li, Z. Xie, Z. Song, Z. Pan, Z. Huang, Z. Xu, Z. Zhang, Z. Zhang, DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning, 2025, arXiv, abs/2501.12948 arXiv:2501.12948, URL: https://api.semanticscholar.org/CorpusID:275789950.

[29] G. Team, P. Georgiev, V.I. Lei, R. Burnell, L. Bai, A. Gulati, G. Tanzer, D. Vincent, Z. Pan, S. Wang, et al., Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context, 2024, arXiv preprint arXiv:2403.05530.

[30] N. Bjørner, L. de Moura, L. Nachmanson, C.M. Wintersteiger, Programming Z3, in: Engineering Trustworthy Software Systems: 4th International School, SETSS 2018, Chongqing, China, April 7–12, 2018, Tutorial Lectures, Springer International Publishing, Cham, 2019, pp. 148–201, http://dx.doi.org/10.1007/978-3-030-17601-3_4.

[31] D. Beyer, Competition on software verification and witness validation: SV-COMP 2023, in: International Conference on Tools and Algorithms for the Construction and Analysis of Systems, Springer, 2023, pp. 495–522.

[32] A. Biere, M. Fleury, F. Pollitt, CaDiCaL vivinst, IsaSAT, Gimsatul, Kissat, and TabularaSAT entering the SAT competition 2023, Proc. SAT Compét. (2023) 14.

[33] A. Biere, T. Faller, K. Fazekas, M. Fleury, N. Froleyks, F. Pollitt, CaDiCaL, Gimsatul, IsaSAT and Kissat entering the SAT competition 2024, Proc. SAT Compét. (2024) 8–10.

[34] B. Selman, D.G. Mitchell, H.J. Levesque, Generating hard satisfiability problems, Artificial Intelligence 81 (1996) 17–29, URL: https://api.semanticscholar.org/CorpusID:14000697.

[35] D.G. Mitchell, H.J. Levesque, Some pitfalls for experimenters with random SAT, Artificial Intelligence 81 (1996) 111–125, URL: https://api.semanticscholar.org/CorpusID:17014913.

[36] T. Madusanka, I. Pratt-Hartmann, R. Batista-Navarro, Natural language satisfiability: Exploring the problem distribution and evaluating transformer-based language models, in: L.-W. Ku, A. Martins, V. Srikumar (Eds.), Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Bangkok, Thailand, 2024, pp. 15278–15294, http://dx.doi.org/10.18653/v1/2024.acl-long.815, URL: https://aclanthology.org/2024.acl-long.815/.

[37] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q.V. Le, D. Zhou, et al., Chain-of-thought prompting elicits reasoning in large language models, Adv. Neural Inf. Process. Syst. 35 (2022) 24824–24837.

[38] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R.K.-W. Lee, E.-P. Lim, Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models, in: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2023, pp. 2609–2634.

[39] X. Wang, J. Wei, D. Schuurmans, Q.V. Le, E.H. Chi, S. Narang, A. Chowdhery, D. Zhou, Self-consistency improves chain of thought reasoning in language models, in: The Eleventh International Conference on Learning Representations, 2022.

[40] X. Ye, Q. Chen, I. Dillig, G. Durrett, Satlm: Satisfiability-aided language models using declarative prompting, Adv. Neural Inf. Process. Syst. 36 (2024).

[41] Y. Zhang, H.-L. Zhen, Z. Pei, Y. Lian, L. Yin, M. Yuan, B. Yu, SoLA: Solver-layer adaption of LLM for better logic reasoning, 2024, arXiv preprint arXiv:2402.11903.

[42] S. Min, X. Lyu, A. Holtzman, M. Artetxe, M. Lewis, H. Hajishirzi, L. Zettlemoyer, Rethinking the role of demonstrations: What makes in-context learning work? in: Y. Goldberg, Z. Kozareva, Y. Zhang (Eds.), Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, 2022, pp. 11048–11064, http://dx.doi.org/10.18653/v1/2022.emnlp-main.759, URL: https://aclanthology.org/2022.emnlp-main.759.

[43] X. Chen, R.A. Chi, X. Wang, D. Zhou, Premise order matters in reasoning with large language models, 2024, arXiv preprint arXiv:2402.08939.